1. INTRODUCTION

Healthcare systems worldwide face increasing challenges due to the rising prevalence of chronic and infectious diseases. Conditions such as diabetes, heart disease, malaria, and pneumonia continue to contribute significantly to global morbidity and mortality. Early detection and timely medical intervention are essential to reduce complications, improve survival rates, and lower overall healthcare costs. However, most conventional diagnostic systems are designed to detect only one disease at a time, requiring multiple tests, tools, and consultations. This fragmented approach often leads to delayed diagnosis, increased expenses, and limited accessibility, particularly in resource-constrained environments.

With the rapid advancement of Artificial Intelligence (AI) and Machine Learning (ML), intelligent diagnostic systems have emerged as powerful tools for healthcare prediction and decision support. Machine learning algorithms can analyze large volumes of clinical and medical data to identify hidden patterns and predict disease risks with high accuracy. Unlike traditional rule-based systems, ML models continuously improve their performance through data-driven learning. Techniques such as Support Vector Machine (SVM), Random Forest, K-Nearest Neighbors (KNN), and ensemble learning methods have shown promising results in disease prediction tasks. Additionally, deep learning models such as Convolutional Neural Networks (CNNs) have proven highly effective in analyzing medical images for detecting diseases like malaria and pneumonia.

This project introduces a Scalable Multi-Disease Prediction System for Early Intervention using Machine Learning, designed to provide a unified platform capable of predicting multiple diseases within a single framework. The system integrates structured clinical data and medical image analysis to generate accurate and reliable risk assessments. By combining multiple models and applying preprocessing techniques such as data cleaning, normalization, and feature selection, the system enhances predictive performance and reduces errors.A key feature of the proposed system is its modular and scalable architecture. The design allows new diseases, datasets, and models to be added without significant modification to the existing structure.

1.1 ORGANIZATION PROFILE

VET Institute of Arts and Science (VETIAS) is a prestigious educational institution that strives to provide a unique and transformative learning experience to its students. As a part of the Vellalar family, VETIAS upholds the same values of academic excellence, community service, and commitment to social justice that the family has been known for over the years.

VETIAS offers a range of academic programs that are designed to foster intellectual curiosity, critical thinking, and creativity among students. The institution places a strong emphasis on collaborative learning, encouraging students to work together in order to develop their ideas and learn from one another. At the same time, individual intellectual development is also encouraged with students being provided with ample opportunities to pursue their own interests and passions.

VETIAS is also committed to promoting diversity and inclusivity, and values the diverse perspectives and experiences that students bring to the institution. By providing a learning environment that respects and celebrates these differences, VETIAS prepares students to become ethical leaders with a truly global perspective.

The institution also recognizes the importance of service to the larger community, and encourages its students and faculty to engage in meaningful service projects that make a positive impact on society. By fostering a sense of social responsibility and civic engagement among its students, VETIAS prepares them to be active and engaged citizens who are committed to making a difference in the world.

In essence, VETIAS is more than just an academic institution. It is a community of scholars, learners, and leaders who are dedicated to excellence, creativity, and service. With a focus on knowledge, understanding, and intellectual courage, VETIAS aims to prepare its students to lead purposeful lives in a challenging and changing world.

1.2 System Specification

The system specification refers to a detailed description of the hardware, software, and network components required for the successful implementation of the Multi-Disease Prediction System.

The hardware specification includes the type, model, and configuration of components such as the processor, RAM, hard disk, and peripherals.

The software specification outlines the operating system, programming language, application software, libraries, and frameworks used in the project.

1.2.1 Hardware Configuration

Processor	:	Intel® Core™ i3 Processor
RAM	:	8 GB DDR4 RAM
Monitor	:	14″ Color Monitor
Hard Disk	:	1 TB
Keyboard	:	Standard 102 Keys
Mouse	:	Scrolling Mouse

1.2.2 Software Specification

Operating System	:	Windows 10
Coding Language	:	Python
Front End	:	HTML, CSS, JavaScript
Backend Framework	:	Django
Libraries Used	:	NumPy, Pandas, OpenCV, TensorFlow, XGBoost, Scikit-learn, Matplotlib.

1.2.2.1 Python

Python is a high-level, interpreted, and general-purpose programming language. It was created by Guido van Rossum in 1991 and has since become one of the most widely used programming languages across the world. Python is well known for its simple syntax, high readability, and ease of use, which makes it suitable for both beginners and advanced developers.

Python is highly versatile and is extensively used in various application domains such as web development, data science, machine learning, image processing, and artificial intelligence. Its flexibility and powerful programming features make it an ideal choice for developing intelligent systems like the proposed keratoconus screening application.

One of the major strengths of Python is its large and active developer community, which provides a vast collection of libraries, frameworks, and tools that can be easily integrated into projects. These libraries enable developers to solve complex problems with minimal code, thereby improving development speed and efficiency.

Python is an interpreted language, meaning the code is executed line by line rather than being compiled into machine-level instructions. This feature simplifies debugging and testing, making Python highly suitable for rapid prototyping, experimentation, and real-time application development.

Python Django

Django is a popular web framework used to build web applications quickly and efficiently using the Python programming language. It follows the Model–View–Controller (MVC) architectural pattern (implemented as Model–View–Template in Django) and is designed to support the rapid development of secure, maintainable, and scalable web applications. Django provides a wide range of built-in tools and libraries that simplify web development. These include an Object-Relational Mapping (ORM) layer that allows developers to interact with databases using Python code, built-in security features to protect against common web vulnerabilities, and a powerful templating engine for creating dynamic and responsive HTML pages.

The framework also supports various third-party libraries and extensions such as Django REST Framework for building RESTful APIs, Django CMS for content management systems, and Django Channels for developing real-time web applications. These features make Django suitable for developing modern and complex web-based systems.

Django's popularity is mainly due to its flexibility, ease of use, and strong community support. Its comprehensive documentation and active developer community make it an excellent choice for both beginners and experienced developers.

Features of Django

- Django is a high-level, open-source web framework built on Python.
- Follows the Model–View–Template (MVT) architectural pattern.
- Provides an Object-Relational Mapping (ORM) system for easy database operations.
- Includes built-in security features such as protection against SQL injection, XSS, and CSRF attacks.
- Offers a powerful templating engine for creating dynamic and responsive web pages.
- Supports rapid application development with reusable and maintainable code.
- Comes with an automatic admin interface for easy application management.
- Highly scalable and suitable for developing large-scale web applications.

Python Django Installation

- Install Python: Django is a Python framework, so you'll need to have Python installed on your computer.
- Install pip: Django is installed using pip, which is a package manager for Python. If you don't already have pip installed you can install pip first.
- Install Django: Once you have Python and pip installed, you can install Django by running the following command in your terminal or command prompt: pip install Django

OpenCV

OpenCV (Open Source Computer Vision Library) is an open-source library used for real-time computer vision and image processing applications. It provides a wide range of functions for image acquisition, processing, feature extraction, and object detection, making it suitable for applications such as medical imaging, surveillance, and pattern recognition.

Features of OpenCV

- Supports image and video processing operations
- Provides tools for feature detection and extraction
- Works efficiently with real-time applications
- Supports multiple programming languages including Python and C++
- Optimized for performance

OpenCV Installation

- Ensure Python is installed on the system
- Install OpenCV using pip - pip install opencv-python

NumPy

NumPy (Numerical Python) is an open-source library used for numerical and scientific computing in Python. It provides support for large multi-dimensional arrays and matrices along with a collection of mathematical functions to operate on them efficiently.

Features of NumPy

- Supports multi-dimensional arrays and matrices
- Provides fast mathematical and statistical operations
- Efficient memory management
- Forms the foundation for many scientific libraries

NumPy Installation

- Install NumPy using pip - pip install numpy

Pandas

Pandas is a powerful open-source Python library used for data manipulation and analysis. It provides data structures such as Series and DataFrames that make handling structured data easy and efficient.

Features of Pandas

- Easy handling of structured and tabular data
- Powerful data filtering and aggregation tools
- Supports data import and export from various formats
- Integrates well with NumPy and machine learning libraries

Pandas Installation

- Install Pandas using pip - pip install pandas

TensorFlow

TensorFlow is an open-source machine learning framework developed by Google Brain Team. It is widely used for building and deploying machine learning and deep learning models, especially neural networks.

Components of TensorFlow

- **TensorFlow Core** – Basic tools for model building and training
- **TensorFlow Keras** – High-level API for deep learning
- **TensorFlow Estimators** – Simplifies model training and evaluation
- **TensorFlow Dataset** – Tools for handling large datasets
- **TensorFlow Lite** – Lightweight version for mobile and embedded devices

Features of TensorFlow

- Flexible and powerful programming model
- Scalable from CPU to GPU and distributed systems
- High performance with hardware acceleration
- Visualization tools for debugging models
- Large and active developer community

TensorFlow Installation

- Ensure Python 3.10 .
- Install TensorFlow using pip - pip install tensorflow

Scikit-learn

Scikit-learn is used for training predictive models on clinical datasets, performing data preprocessing, feature selection, and evaluating model accuracy.

Features of Scikit-learn

- Supports classification, regression, and clustering algorithms.
- Provides tools for preprocessing and model evaluation.
- Simple and consistent API for easy use.
- Works well with NumPy and Pandas.

Scikit-learn Installation

- Install Scikit-learn using pip scikit-learn

XGBoost

XGBoost improves prediction accuracy through ensemble learning and is widely used in machine learning competitions and real-world applications.

Features of XGBoost

- Provides high prediction accuracy for disease classification tasks.
- Handles large clinical datasets efficiently.
- Reduces overfitting through regularization techniques.
- Enhances performance using ensemble learning methods.

XGBoost Installation

- Install XGBoost using pip- pip install xgboost

Matplotlib

Matplotlib is a Python plotting library used for data visualization. It allows developers to create static, animated, and interactive graphs and charts.

Features of Matplotlib

- Supports various types of plots and graphs
- Easy customization of visualizations
- Works well with NumPy and Pandas
- Widely used for scientific visualization

Matplotlib Installation

- Install Matplotlib using pip- pip install matplotlib

1.2.2.2 Web Designing

HTML

HTML (HyperText Markup Language) is a standard markup language used for creating web pages and web applications. It is used to structure the content on the web and define how text, images, videos, and other multimedia elements are displayed in a browser. HTML uses a system of tags and attributes to describe the structure and presentation of content.

HTML documents are made up of a series of elements enclosed within angle brackets. Each element generally consists of a start tag, content, and an end tag. The start tag specifies the beginning of an element, while the end tag indicates its termination. HTML elements can also include attributes, which provide additional information such as styling, identification, or behavior of elements.

CSS

CSS (Cascading Style Sheets) is a style sheet language used to describe the visual appearance and layout of web pages written in HTML or XHTML. It allows developers to separate the presentation layer from the content, making web pages easier to maintain and update.

CSS is used to control various design aspects such as colors, fonts, spacing, alignment, and layout of web elements.

Styles can be applied using external CSS files, internal style sheets, or inline styles, depending on the design requirements.

This separation of design from content improves flexibility, consistency, and efficiency in web development, and also helps in creating responsive and visually appealing user interfaces.

JavaScript

JavaScript is a simple and powerful programming language mainly used to make web pages interactive. It allows websites to respond to user actions such as button clicks, form submissions, and mouse movements without reloading the page. Using JavaScript, developers can display alerts, validate forms, update content dynamically, and create animations, which improves the overall user experience.

JavaScript was developed by Brendan Eich in 1995 and is supported by all major web browsers such as Chrome, Firefox, Safari, and Edge. This wide support makes JavaScript a standard language for web development. It works together with HTML and CSS to control the behavior and appearance of web pages.

In addition to running in web browsers, JavaScript is also used on the server side and in mobile application development. Its easy syntax, flexibility, and large number of available libraries make it suitable for beginners as well as professional developers. Overall, JavaScript plays an important role in building modern, user-friendly web applications.

2. SYSTEM STUDY

2.1 EXISTING SYSTEM

The existing healthcare diagnostic systems primarily focus on identifying one disease at a time. For each medical condition, separate diagnostic tools, tests, and predictive models are required. This isolated approach makes the overall diagnosis process time-consuming and less efficient.

Most traditional systems depend heavily on manual medical examinations, laboratory reports, and individual clinical assessments. While these methods are reliable, they often require significant time, specialized equipment, and expert involvement, which may not always be easily accessible.

In many cases, existing prediction systems are built using single machine learning models without integration or ensemble techniques. This can limit prediction accuracy and reduce the system's ability to handle complex or large-scale healthcare data effectively.

As a result, early detection of multiple diseases becomes challenging, especially when symptoms overlap. The lack of an integrated and scalable platform increases healthcare costs, delays diagnosis, and reduces the effectiveness of preventive healthcare strategies.

2.1.1 Drawbacks of Existing System

- **High Cost:** Diagnostic equipment and multiple testing procedures increase overall healthcare expenses.
- **Single Disease Detection:** Most systems focus on only one disease at a time, lacking integrated analysis.
- **Time Consuming:** Separate tests and manual evaluation lead to delays in diagnosis.
- **Limited Scalability:** Existing systems are difficult to expand for additional diseases or larger datasets.

2.2 PROPOSED SYSTEM

The proposed system introduces a Scalable Multi-Disease Prediction System that integrates the prediction of multiple diseases within a single unified platform. Unlike traditional systems that focus on one condition at a time, this system allows users to assess risks for diseases such as diabetes, heart disease, malaria, and pneumonia through a common interface.

The system utilizes machine learning and deep learning techniques to analyze both clinical data and medical images. Algorithms such as SVM, Random Forest, KNN, and XGBoost are used for structured data prediction, while Convolutional Neural Networks (CNNs) are implemented for image-based disease detection. Ensemble learning methods are applied to improve prediction accuracy and reliability.

The platform is developed using a web-based framework, enabling easy access and real-time prediction. Its modular and scalable design allows the addition of new diseases and datasets without major structural changes. This approach supports early detection, reduces diagnostic delays, and enhances preventive healthcare management.

2.2.1 Features of Proposed System

- **Integrated Multi-Disease Prediction:**Provides prediction for multiple diseases within a single unified platform.
- **High Accuracy:**Uses advanced machine learning and ensemble techniques to improve prediction performance.
- **Image-Based Detection:**Supports deep learning models for analyzing medical images such as malaria and pneumonia scans.
- **Scalable Architecture:**Allows easy addition of new diseases and datasets without major system changes.
- **User-Friendly Interface:**Web-based platform that enables real-time and easy access for users.

3. SYSTEM DESIGN AND DEVELOPMENT

3.1 FILE DESIGN

The file design of a project refers to the organization and structure of the project files, including naming conventions, directory hierarchy, and the relationship between files. Proper file design helps maintain the project in a systematic manner, making the system easier to develop, manage, and update. In the multi-disease prediction system, different files are used to handle user requests, process medical data, and generate disease predictions using machine learning models.

manage.py

This file is a command-line utility used to manage the Django project. It allows the developer to run the development server, create and apply database migrations, and perform other administrative tasks related to the project.

urls.py

This file defines the URL patterns for the web application. It connects the user requests from the browser to the appropriate views that process the request and return the disease prediction results.

settings.py

This file contains all the configuration settings of the Django project. It includes information about installed applications, database configuration, templates, static files, and other environment settings required for the system.

views.py

This file contains the main application logic. It receives the user input such as health parameters (age, blood pressure, glucose level, etc.), processes the data, sends it to the machine learning model, and returns the predicted disease result to the user.

model.py

This file contains the trained machine learning models used for disease prediction. Algorithms such as Random Forest, Support Vector Machine, or XGBoost are used to analyze the input health parameters and predict diseases such as diabetes, heart disease, or other medical conditions.

prediction.py

This file handles the prediction process. It loads the trained model and processes the user input data to generate the final disease prediction output.

index.html

This file contains the user interface of the application designed using HTML, CSS, and JavaScript. It allows users to enter their health details and submit the information to get the disease prediction result.

result.html

This file displays the prediction results generated by the system. It shows whether the user is at risk of a particular disease based on the input parameters.

3.2 INPUT DESIGN

Input design refers to the process of collecting data from users for system processing. In the multi-disease prediction system, users provide health parameters such as age, glucose level, blood pressure, cholesterol, and BMI for predicting diabetes and heart disease, and upload medical images such as blood smear images or chest X-ray images for detecting malaria and pneumonia. The system validates the inputs and sends the data to machine learning models to analyze and predict the presence of these diseases. Additionally, the input design ensures that the data entered by users is accurate, complete, and in the correct format before processing, while user-friendly interfaces such as forms and upload options make data entry simple and efficient. Proper validation techniques help reduce errors and improve the reliability of predictions, thereby enhancing the overall performance and accuracy of the system.

3.3 OUTPUT DESIGN

Output design refers to the way the system presents the processed results to the user in a clear and understandable format. In the multi-disease prediction system, the output displays the prediction results based on the user's input data and uploaded medical images. The system shows whether the person is likely to have diabetes, heart disease, malaria, or pneumonia, along with the prediction status such as positive or negative. The results are displayed on the web interface in a simple and user-friendly manner so that users can easily understand the health prediction outcome.

3.4 SYSTEM DEVELOPMENT

System development refers to the process of designing, implementing, and integrating different components of the keratoconus detection system. The system is developed using Python, Django, OpenCV, and machine learning techniques to analyze corneal reflection patterns and detect keratoconus. The development process includes image capture, preprocessing, feature extraction, and classification.

3.4.1 Description of Modules

- User Interface Module
- Data Processing Module
- Disease Prediction Module
- Result Display Module
- Database Management Module

User Interface Module

This module provides the interaction platform between the user and the system. It allows users to enter their health parameters such as age, glucose level, blood pressure, and cholesterol. Users can also upload medical images required for malaria and pneumonia detection. The interface is designed using HTML, CSS, and JavaScript for easy usability. This module ensures that users can easily access the system and provide the required inputs.

Data Processing Module

This module is responsible for processing the data provided by the user. It validates the entered health parameters to ensure accuracy and correctness. The module also preprocesses the uploaded medical images before analysis. Data cleaning and formatting are performed to make the data suitable for the machine learning models. This helps improve the reliability and accuracy of the disease prediction.

Disease Prediction Module

This module performs the core functionality of the system. It uses machine learning algorithms to analyze the processed data. The module predicts diseases such as diabetes, heart disease, malaria, and pneumonia. Trained models evaluate the input data and generate prediction results. This module plays a key role in providing accurate disease detection.

Result Display Module

This module is responsible for presenting the prediction results to the user. The system displays whether the user is likely to have a particular disease or not. Results are shown in a simple and understandable format on the web interface. It helps users easily interpret the prediction outcome. This module improves user experience by clearly communicating the system output.

Database Management Module

This module manages the storage of user information and prediction results. It stores the health parameters and uploaded data in the system database. The module helps maintain records for future reference and analysis. It also supports efficient data retrieval when needed. Proper database management ensures the security and organization of system data.

4. SYSTEM TESTING AND IMPLEMENTATION

System Testing

System testing is the process of evaluating the complete system to ensure that it functions correctly and meets the specified requirements. In the multi-disease prediction system, testing is performed to verify that all modules work properly and produce accurate results. The system is tested by providing different health parameters and medical images to check the prediction of diseases such as diabetes, heart disease, malaria, and pneumonia. This testing helps identify and fix errors, ensuring the reliability and performance of the application before deployment.

Types of Testing

A. Unit Testing
B. Integration Testing
C. System Testing
D. User Acceptance Testing
E. Performance Testing
F. Validation Testing

A. Unit Testing

Unit testing is used to test individual modules of the system separately. Each component such as input processing, data preprocessing, and prediction module is tested independently. This helps identify errors in specific parts of the system. It ensures that every module works correctly before integration.

B. Integration Testing

Integration testing checks the interaction between different modules of the system. It ensures that the data flows properly from the user interface to the processing and prediction modules. This testing verifies that all modules work together without errors. It helps detect issues in module communication.

C. System Testing

System testing evaluates the complete system to ensure it meets the project requirements. It verifies that all components function together correctly. The system is tested using different health parameters and medical images. This ensures accurate prediction of diseases such as diabetes, heart disease, malaria, and pneumonia.

D. User Interface Testing

User acceptance testing is performed to confirm that the system meets user requirements. End users test the application to ensure it is easy to use and understand. It verifies whether the system produces correct prediction results. This testing ensures user satisfaction before system deployment.

E. Performance Testing

Performance testing is conducted to evaluate how efficiently the system performs under different conditions. It measures the response time and speed of generating prediction results. The system is tested with multiple user inputs to check its ability to handle workload without delays. This testing ensures that the application runs smoothly even when many requests are processed at the same time.

F. Validation Testing

Validation testing is performed to ensure that the system produces accurate and reliable prediction results. It verifies whether the developed system meets the required objectives and user requirements. In the multi-disease prediction system, the prediction models are tested using different health parameters and medical images. The output generated by the system is compared with the expected or actual results. This helps to check the correctness of the prediction models. Validation testing improves the reliability and accuracy of the disease prediction process.

IMPLEMENTATION

Implementation is the process of converting the system design into a fully functional application. In the multi-disease prediction system, the implementation involves developing the web interface, integrating machine learning models, and connecting different modules of the system. The application is developed using Python and the Django framework for the backend, while HTML, CSS, and JavaScript are used to design the user interface.

During implementation, the system collects user health parameters and medical images through the web interface. The input data is processed and analyzed using trained machine learning models to predict diseases such as diabetes, heart disease, malaria, and pneumonia. The prediction results are then displayed to the user through the web application in a clear and understandable format.

The stage consists of the following steps:

1. Collecting medical datasets and images for diabetes, heart disease, malaria, and pneumonia prediction.
2. Cleaning and preparing the collected data for machine learning models.
3. Training machine learning algorithms using the prepared datasets.
4. Testing the trained models to check accuracy and reliability.
5. Processing user input and displaying the predicted disease results.

5.CONCLUSION

The multi-disease prediction system is developed to help in the early detection of diseases such as diabetes, heart disease, malaria, and pneumonia using machine learning techniques. The system collects health parameters and medical images from the user through a web-based interface. These inputs are processed and analyzed using trained machine learning models. The system then predicts the possibility of different diseases based on the given data. The results are displayed clearly to the user through the application interface. This approach helps in providing faster and more efficient disease prediction.

The developed system can assist healthcare professionals and individuals in identifying potential health risks at an early stage. It helps reduce the time required for preliminary diagnosis and supports quick decision-making. The system is designed to be simple, user-friendly, and easily accessible through a web application. By using machine learning algorithms, the system improves the accuracy of disease prediction. It also helps in spreading awareness about health conditions among users. Overall, the multi-disease prediction system demonstrates the importance of technology in improving healthcare services.

6.FUTURE ENHANCEMENT

Improved Accuracy

One of the important areas for improvement in the multi-disease prediction system is prediction accuracy. By training the models with larger and more diverse medical datasets, the system can learn more complex disease patterns. Advanced machine learning and deep learning algorithms can also be used to improve prediction performance. This will help reduce incorrect predictions for diseases such as diabetes, heart disease, malaria, and pneumonia.

Security Improvement

Security is an important factor when handling patient health information and medical data. The system can be enhanced by implementing secure data storage and encrypted communication methods. This will help protect sensitive user information from unauthorized access. Proper security measures will ensure safe and reliable use of the application.

Integration with Hospital Systems

In the future, the system can be integrated with hospital databases and electronic health record systems. This will allow the system to access real-time patient data for better prediction. Doctors can use the system as a supporting tool for faster diagnosis. This integration will improve the efficiency and usefulness of the system in healthcare environments.

Mobile Application Support

The system can also be developed as a mobile application in the future. This will allow users to easily enter their health details or upload medical images through smartphones. The mobile application can quickly analyze the data and provide disease prediction results. This will make the system more accessible and convenient for users.

BIBLIOGRAPHY

References

Journal References:

1. Machine Learning-Based Disease Prediction System Using Health Data – This paper discusses the use of machine learning algorithms to predict diseases using patient health parameters and medical datasets.
2. Heart Disease Prediction Using Machine Learning Techniques – This journal explains different machine learning methods used to predict heart disease based on clinical data.
3. Diabetes Prediction Using Machine Learning Algorithms – This study focuses on using supervised learning models to detect diabetes using patient health records.
4. Malaria Detection Using Machine Learning and Image Processing – This paper describes the use of image processing and machine learning techniques to detect malaria from blood smear images.
5. Pneumonia Detection from Chest X-ray Images Using Deep Learning – This journal presents deep learning methods for identifying pneumonia using chest X-ray images.

Books:

1. Machine Learning – Tom M. Mitchell, McGraw-Hill Education.
2. Hands-On Machine Learning – Aurélien Géron, O'Reilly Media.
3. Deep Learning – Ian Goodfellow, Yoshua Bengio, Aaron Courville, MIT Press.
4. Learning Python – Mark Lutz, O'Reilly Media.

Websites:

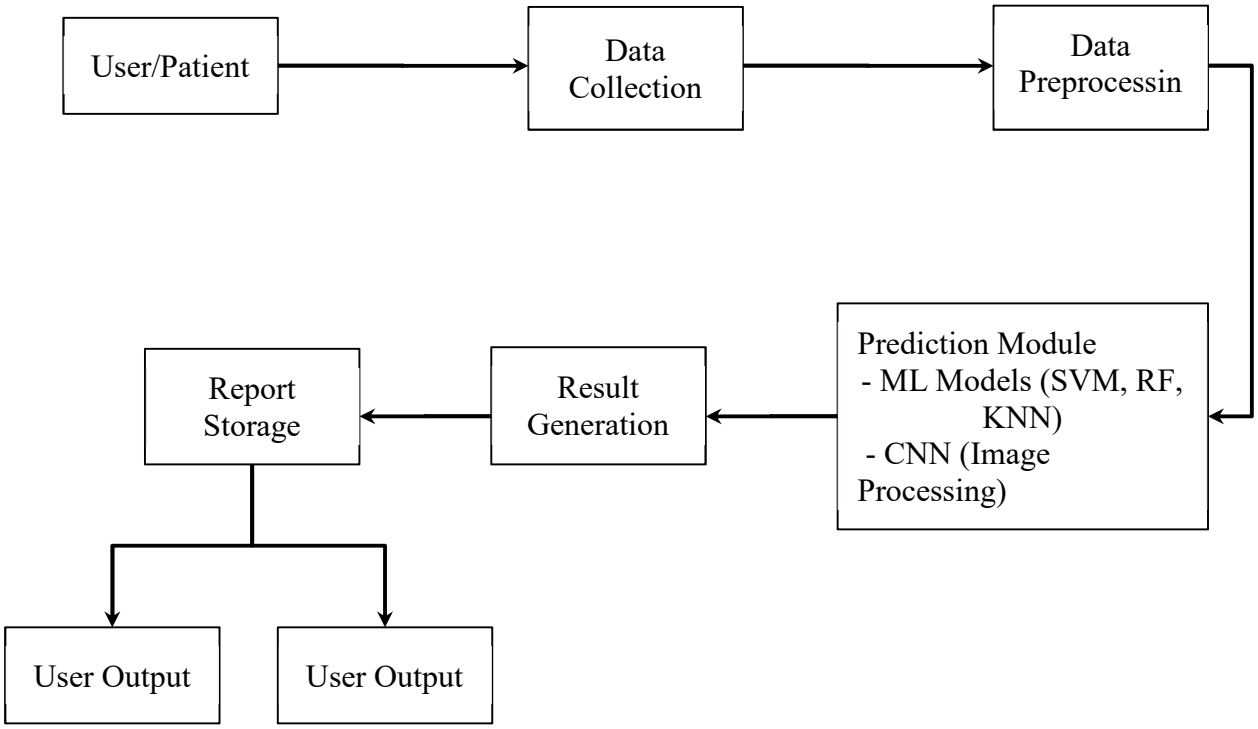
1. Scikit-learn – https://scikit-learn.org
2. TensorFlow – https://www.tensorflow.org
3. Django – https://www.djangoproject.com
4. Kaggle – https://www.kaggle.com

APPENDICES

A. DATA FLOW DIAGRAM

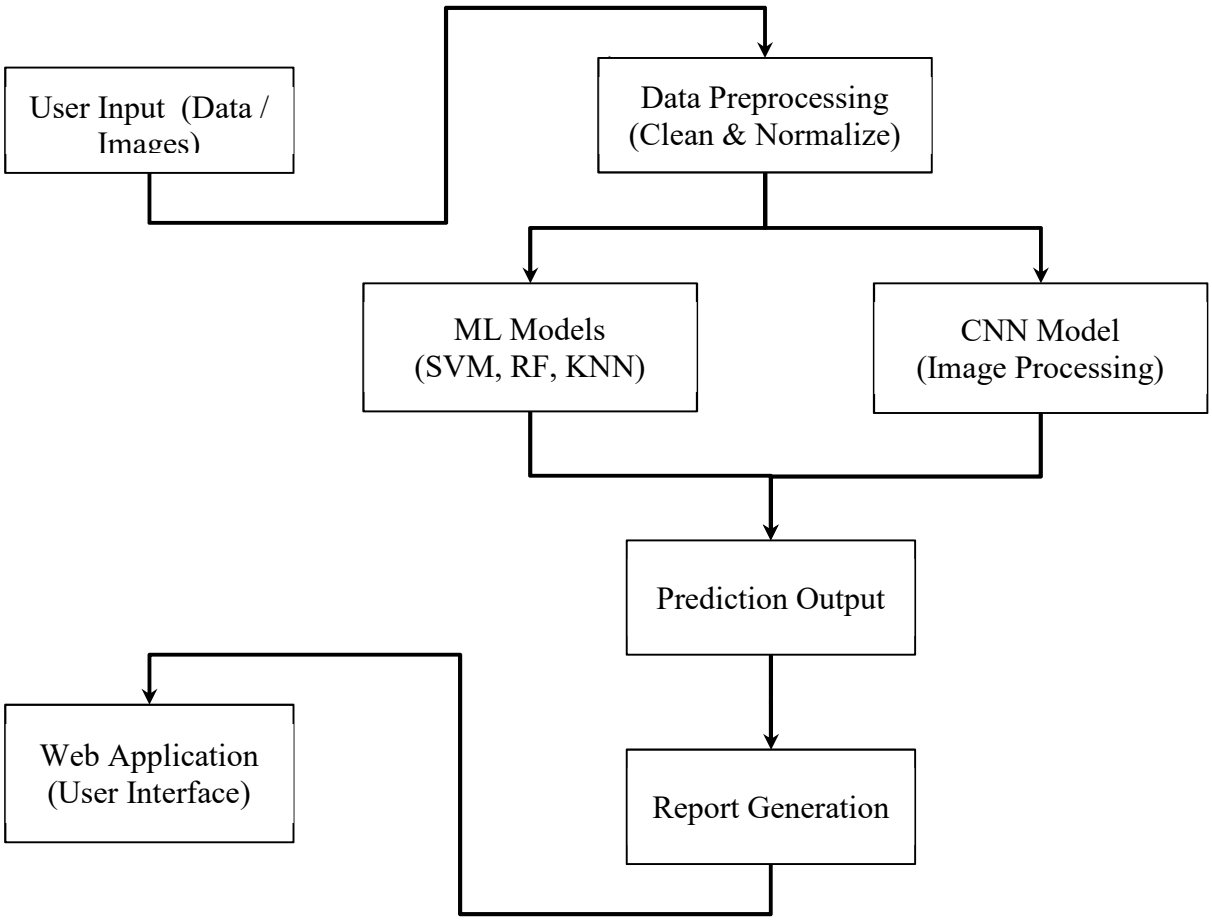
The Data Flow Diagram (DFD) represents the flow of data within the Scalable Multi-Disease Prediction System. It illustrates how patient data is collected, processed, and transformed into meaningful disease predictions and health reports. The system integrates both machine learning models and deep learning techniques to analyze clinical data and medical images.

DATA FLOW



COMPOUND DIAGRAM

The compound diagram represents the overall working process of the multi-disease prediction system. It shows how the system takes input, processes it, uses different models, and produces the final output.



B. SAMPLE CODING

manage.py

```
import os
import sys
def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'multidisease.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)
if __name__ == '__main__':
    main()
```

urls.py

```
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static
from . import views
urlpatterns = [
    path('', views.index, name='index'),
    path('about/', views.about, name='about'),
```

```
    path('services/', views.services, name='services'),
    path('contact/', views.contact, name='contact'),
    path('history/', views.history, name='history'),
    path('diabetes/', views.diabetes, name='diabetes'),
    path('diabetes/predict/', views.diabetes_predict, name='diabetes_predict'),
    path('heart-disease/', views.heart_disease, name='heart_disease'),
    path('heart-predict/', views.heart_disease_predict, name='heart_disease_predict'),
    path('malaria/', views.malaria, name='malaria'),
    path('malaria/predict/', views.malaria_predict, name='malaria_predict'),
    path('pneumonia/', views.pneumonia, name='pneumonia'),
    path('pneumonia/predict/', views.pneumonia_predict, name='pneumonia_predict'),
    path('api/diabetes/', views.api_diabetes, name='api_diabetes'),
    path('api/heart-disease/', views.api_heart_disease, name='api_heart_disease'),
]
if settings.DEBUG:
urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

settings.py

```
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent
SECRET_KEY = 'django-insecure-%xt+!)ir_qt+%ne$9u5c6mbe+nui=!y_%aan(*)!h81zn8u0#w'
DEBUG = True
ALLOWED_HOSTS = []
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
```

```
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'predictor',
]
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
ROOT_URLCONF = 'multidisease.urls'
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
```

```
        },
    },
]
WSGI_APPLICATION = 'multidisease.wsgi.application'
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
LANGUAGE_CODE = 'en-us'
```

```
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True
STATIC_URL = 'static/'
MEDIA_URL = 'media/'
MEDIA_ROOT = BASE_DIR / 'media'
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

view.py

```
from django.shortcuts import render, redirect
from django.http import JsonResponse, HttpResponse
from django.views.decorators.csrf import csrf_exempt
from django.contrib import messages
from .models import (
    PredictionHistory, DiabetesPrediction, HeartDiseasePrediction,
    MalariaPrediction, PneumoniaPrediction
)
from .ml_models import (
    DiabetesModel, HeartDiseaseModel, MalariaModel, PneumoniaModel,
    get_diabetes_sample_data, get_heart_disease_sample_data
)
import numpy as np
import os
diabetes_model = DiabetesModel()
heart_disease_model = HeartDiseaseModel()
malaria_model = MalariaModel()
pneumonia_model = PneumoniaModel()
def index(request):
```

```
    return render(request, 'predictor/index.html')
def about(request):
    return render(request, 'predictor/about.html')
def services(request):
    return render(request, 'predictor/services.html')
def diabetes(request):
    return render(request, 'predictor/diabetes.html')
def heart_disease(request):
    return render(request, 'predictor/heart_disease.html')
def malaria(request):
    return render(request, 'predictor/malaria.html')
def pneumonia(request):
    return render(request, 'predictor/pneumonia.html')
def diabetes_predict(request):
    if request.method == 'POST':
    try:
        features = [
        float(request.POST.get('pregnancies', 0)),
        float(request.POST.get('glucose', 0)),
        float(request.POST.get('blood_pressure', 0)),
        float(request.POST.get('skin_thickness', 0)),
        float(request.POST.get('insulin', 0)),
        float(request.POST.get('bmi', 0)),
        float(request.POST.get('diabetes_pedigree', 0)),
        float(request.POST.get('age', 0))
        ]
        X, y = get_diabetes_sample_data()
```

```
diabetes_model.train(X, y)
result = diabetes_model.predict(features)
  diabetes_pred = DiabetesPrediction.objects.create(
   pregnancies=int(features[0]),
   glucose=features[1],
   blood_pressure=features[2],
   skin_thickness=features[3],
   insulin=features[4],
   bmi=features[5],
   diabetes_pedigree=features[6],
   age=int(features[7]),
   result=result['prediction']
)
   PredictionHistory.objects.create(
   disease_type='Diabetes',
   input_data={
      'pregnancies': features[0],
      'glucose': features[1],
      'blood_pressure': features[2],
      'skin_thickness': features[3],
      'insulin': features[4],
      'bmi': features[5],
      'diabetes_pedigree': features[6],
      'age': features[7]
   },
   prediction_result=result['prediction'],
   confidence_score=result['confidence']
```

```
            )
            context = {
                'result': result,
                'features': {
                    'Pregnancies': features[0],
                    'Glucose': features[1],
                    'Blood Pressure': features[2],
                    'Skin Thickness': features[3],
                    'Insulin': features[4],
                    'BMI': features[5],
                    'Diabetes Pedigree': features[6],
                    'Age': features[7]
                }
            }
            return render(request, 'predictor/diabetes_result.html', context)
        except Exception as e:
            messages.error(request, f"Error in prediction: {str(e)}")
            return redirect('diabetes')
    return redirect('diabetes')
def heart_disease_predict(request):
    if request.method == "POST":
        try:
            age = int(request.POST.get('age'))
            sex = int(request.POST.get('sex'))
            cp = int(request.POST.get('cp'))
            trestbps = float(request.POST.get('trestbps'))
            chol = float(request.POST.get('chol'))
```

```
fbs = int(request.POST.get('fbs'))
restecg = int(request.POST.get('restecg'))
thalach = float(request.POST.get('thalach'))
exang = int(request.POST.get('exang'))
oldpeak = float(request.POST.get('oldpeak'))
slope = int(request.POST.get('slope'))
ca = int(request.POST.get('ca'))
thal = int(request.POST.get('thal'))
features = [
    age, sex, cp, trestbps, chol,
    fbs, restecg, thalach, exang,
    oldpeak, slope, ca, thal
]
X, y = get_heart_disease_sample_data()
heart_disease_model.train(X, y)
result = heart_disease_model.predict(features)
HeartDiseasePrediction.objects.create(
    age=age,
    sex=sex,
    cp=cp,
    trestbps=trestbps,
    chol=chol,
    fbs=fbs,
    restecg=restecg,
    thalach=thalach,
    exang=exang,
    oldpeak=oldpeak,
```

```
    slope=slope,
    ca=ca,
    thal=thal,
    result=result['prediction']
)
context = {
    "result": result,
    "features": {
        "Age": age,
        "Sex": "Male" if sex == 1 else "Female",
        "Chest Pain Type": cp,
        "Resting Blood Pressure": trestbps,
        "Serum Cholesterol": chol,
        "Fasting Blood Sugar": fbs,
        "Resting ECG": restecg,
        "Max Heart Rate": thalach,
        "Exercise Induced Angina": exang,
        "ST Depression": oldpeak,
        "Slope": slope,
        "Major Vessels": ca,
        "Thalassemia": thal
    },
    "age": age,
    "sex": "Male" if sex == 1 else "Female",
    "chol": chol,
    "bp": trestbps
}
```

```
            return render(request, "predictor/heart_result.html", context)
        except Exception as e:
            messages.error(request, f"Prediction Error: {str(e)}")
            return redirect("heart_disease")
    return redirect("heart_disease")
@csrf_exempt
def malaria_predict(request):
        if request.method == 'POST' and request.FILES.get('image'):
        try:
            from PIL import Image
            import io
            image_file = request.FILES['image']
            image = Image.open(image_file)
            malaria_pred = MalariaPrediction.objects.create(
                image=image_file,
                result="Processing..."
            )
            result = malaria_model.predict(image)
            malaria_pred.result = result['prediction']
            malaria_pred.confidence = result['confidence']
            malaria_pred.save()
            PredictionHistory.objects.create(
                disease_type='Malaria',
                input_data={'image_name': image_file.name},
                prediction_result=result['prediction'],
                confidence_score=result['confidence']
            )
```

```
            context = {
                'result': result,
                'image_url': malaria_pred.image.url
            }
            return render(request, 'predictor/malaria_result.html', context)
        except Exception as e:
            messages.error(request, f"Error in prediction: {str(e)}")
            return redirect('malaria')
    return redirect('malaria')
@csrf_exempt
def pneumonia_predict(request):
        if request.method == 'POST' and request.FILES.get('image'):
        try:
            from PIL import Image
            import io
            image_file = request.FILES['image']
            image = Image.open(image_file)
            pneumonia_pred = PneumoniaPrediction.objects.create(
                image=image_file,
                result="Processing..."
            )
            result = pneumonia_model.predict(image)
            pneumonia_pred.result = result['prediction']
            pneumonia_pred.confidence = result['confidence']
            pneumonia_pred.save()
            PredictionHistory.objects.create(
                disease_type='Pneumonia',
```

```
            input_data={'image_name': image_file.name},
            prediction_result=result['prediction'],
            confidence_score=result['confidence']
        )
        context = {
            'result': result,
            'image_url': pneumonia_pred.image.url
        }
        return render(request, 'predictor/pneumonia_result.html', context)
    except Exception as e:
        messages.error(request, f"Error in prediction: {str(e)}")
        return redirect('pneumonia')
    return redirect('pneumonia')
def history(request):
    predictions = PredictionHistory.objects.all()[:20]
    return render(request, 'predictor/history.html', {'predictions': predictions})
def contact(request):
    return render(request, 'predictor/contact.html')
def api_diabetes(request):
    if request.method == 'POST':
    import json
    try:
        data = json.loads(request.body)
        features = [
            data.get('pregnancies', 0),
            data.get('glucose', 0),
            data.get('blood_pressure', 0),
```

```
            data.get('skin_thickness', 0),
            data.get('insulin', 0),
            data.get('bmi', 0),
            data.get('diabetes_pedigree', 0),
            data.get('age', 0)
        ]
            X, y = get_diabetes_sample_data()
        diabetes_model.train(X, y)
            result = diabetes_model.predict(features)
        return JsonResponse(result)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=400)
return JsonResponse({'error': 'Invalid request method'}, status=405)
def api_heart_disease(request):
    """API endpoint for heart disease prediction"""
    if request.method == 'POST':
        import json
        try:
            data = json.loads(request.body)
            features = [
                data.get('age', 0),
                data.get('sex', 0),
                data.get('cp', 0),
                data.get('trestbps', 0),
                data.get('chol', 0),
                data.get('fbs', 0),
                data.get('restecg', 0),
```

```
            data.get('thalach', 0),
            data.get('exang', 0),
            data.get('oldpeak', 0),
            data.get('slope', 0),
            data.get('ca', 0),
            data.get('thal', 0)
        ]
        X, y = get_heart_disease_sample_data()
        heart_disease_model.train(X, y)
        result = heart_disease_model.predict(features)
        return JsonResponse(result)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=400)
return JsonResponse({'error': 'Invalid request method'}, status=405)
```

model.py

```
from django.db import models
from django.contrib.auth.models import User
class PredictionHistory(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, null=True, blank=True)
    disease_type = models.CharField(max_length=100)
    input_data = models.JSONField()
    prediction_result = models.CharField(max_length=100)
    confidence_score = models.FloatField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    class Meta:
        ordering = ['-created_at']
```

```
    def __str__(self):
        return f"{self.disease_type} - {self.prediction_result} - {self.created_at}"
class DiabetesPrediction(models.Model):
    pregnancies = models.IntegerField()
    glucose = models.FloatField()
    blood_pressure = models.FloatField()
    skin_thickness = models.FloatField()
    insulin = models.FloatField()
    bmi = models.FloatField()
    diabetes_pedigree = models.FloatField()
    age = models.IntegerField()
    result = models.CharField(max_length=50, null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return f"Diabetes Prediction - {self.result}"
class HeartDiseasePrediction(models.Model):
    """Model for heart disease prediction input data"""
    age = models.IntegerField()
    sex = models.IntegerField()
    cp = models.IntegerField()
    trestbps = models.FloatField()
    chol = models.FloatField()
    fbs = models.IntegerField()
    restecg = models.IntegerField()
    thalach = models.FloatField()
    exang = models.IntegerField()
    oldpeak = models.FloatField()
```

```
    slope = models.IntegerField()
    ca = models.IntegerField()
    thal = models.IntegerField()
    result = models.CharField(max_length=50, null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return f"Heart Disease Prediction - {self.result}"
class MalariaPrediction(models.Model):
    image = models.ImageField(upload_to='malaria_images/')
    result = models.CharField(max_length=50, null=True, blank=True)
    confidence = models.FloatField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return f"Malaria Prediction - {self.result}"
class PneumoniaPrediction(models.Model):
    """Model for pneumonia prediction from chest X-ray images"""
    image = models.ImageField(upload_to='pneumonia_images/')
    result = models.CharField(max_length=50, null=True, blank=True)
    confidence = models.FloatField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return f"Pneumonia Prediction - {self.result}"
```

train_models.py

```
import os
import sys
import numpy as np
```

```
import pandas as pd

from PIL import Image

import warnings

warnings.filterwarnings('ignore')

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'multidisease.settings')

import django

django.setup()

from predictor.ml_models import MalariaModel, PneumoniaModel, HeartDiseaseModel, DiabetesModel

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

MALARIA_DIR = os.path.join(BASE_DIR, 'datasets', 'malaria', 'testing_set')

PNEUMONIA_DIR = os.path.join(BASE_DIR, 'datasets', 'pneumonia')

OUTPUT_DIR = os.path.join(BASE_DIR, 'datasets')

def load_malaria_images(image_size=(128, 128)):

    """Load malaria cell images for training"""

    print("Loading Malaria dataset...")

    X = []

    y = []

      parasitized_dir = os.path.join(MALARIA_DIR, 'Parasitized')

    if os.path.exists(parasitized_dir):

        parasitized_files = [f for f in os.listdir(parasitized_dir) if f.endswith('.png')]

        print(f"  Loading {len(parasitized_files)} parasitized images...")

        for i, filename in enumerate(parasitized_files):

            try:

                img_path = os.path.join(parasitized_dir, filename)

                img = Image.open(img_path).convert('RGB')

                img = img.resize(image_size)
```

```
            img_array = np.array(img) / 255.0
            X.append(img_array)
            y.append(1)
            if (i + 1) % 100 == 0:
                print(f"    Loaded {i + 1} parasitized images...")
        except Exception as e:
            print(f"    Error loading {filename}: {e}")
    uninfected_dir = os.path.join(MALARIA_DIR, 'Uninfected')
if os.path.exists(uninfected_dir):
    uninfected_files = [f for f in os.listdir(uninfected_dir) if f.endswith('.png')]
    print(f"  Loading {len(uninfected_files)} uninfected images...")
    for i, filename in enumerate(uninfected_files):
        try:
            img_path = os.path.join(uninfected_dir, filename)
            img = Image.open(img_path).convert('RGB')
            img = img.resize(image_size)
            img_array = np.array(img) / 255.0
            X.append(img_array)
            y.append(0)
            if (i + 1) % 100 == 0:
                print(f"    Loaded {i + 1} uninfected images...")
        except Exception as e:
            print(f"    Error loading {filename}: {e}")
X = np.array(X)
y = np.array(y)
print(f"  Total malaria samples: {len(X)}")
print(f"  Parasitized: {np.sum(y == 1)}, Uninfected: {np.sum(y == 0)}")
```

```
    return X, y
def load_pneumonia_images(image_size=(128, 128)):
    """Load pneumonia chest X-ray images for training"""
    print("\nLoading Pneumonia dataset...")
    X = []
    y = []
    normal_dir = os.path.join(PNEUMONIA_DIR, 'NORMAL')
    if os.path.exists(normal_dir):
        normal_files = [f for f in os.listdir(normal_dir) if f.lower().endswith(('.png',
'.jpg', '.jpeg'))]
        print(f"  Loading {len(normal_files)} normal X-ray images...")
        for i, filename in enumerate(normal_files):
            try:
                img_path = os.path.join(normal_dir, filename)
                img = Image.open(img_path).convert('RGB')
                img = img.resize(image_size)
                img_array = np.array(img) / 255.0
                X.append(img_array)
                y.append(0)
                if (i + 1) % 100 == 0:
                    print(f"    Loaded {i + 1} normal images...")
            except Exception as e:
                print(f"    Error loading {filename}: {e}")
    pneumonia_dir = os.path.join(PNEUMONIA_DIR, 'PNEUMONIA')
    if os.path.exists(pneumonia_dir):
        pneumonia_files = [f for f in os.listdir(pneumonia_dir) if
f.lower().endswith(('.png', '.jpg', '.jpeg'))]
        print(f"  Loading {len(pneumonia_files)} pneumonia X-ray images...")
```

```
        for i, filename in enumerate(pneumonia_files):
            try:
                img_path = os.path.join(pneumonia_dir, filename)
                img = Image.open(img_path).convert('RGB')
                img = img.resize(image_size)
                img_array = np.array(img) / 255.0
                X.append(img_array)
                y.append(1)
                if (i + 1) % 100 == 0:
                    print(f"    Loaded {i + 1} pneumonia images...")
            except Exception as e:
                print(f"    Error loading {filename}: {e}")
    X = np.array(X)
    y = np.array(y)
    print(f"  Total pneumonia samples: {len(X)}")
    print(f"  Normal: {np.sum(y == 0)}, Pneumonia: {np.sum(y == 1)}")
    return X, y
def create_heart_disease_csv():
        print("\nCreating Heart Disease dataset...")
    data = [
        [52, 1, 0, 125, 212, 0, 1, 168, 0, 0, 2, 2, 3, 1],
        [53, 1, 0, 140, 203, 1, 0, 155, 1, 3, 2, 3, 3, 1],
        [70, 1, 0, 160, 289, 0, 0, 171, 0, 0, 2, 3, 7, 1],
        [61, 1, 0, 134, 234, 0, 1, 145, 1, 2, 2, 3, 3, 1],
        [58, 1, 0, 130, 197, 0, 1, 131, 1, 2, 3, 3, 6, 1],
        [44, 1, 2, 130, 219, 0, 1, 188, 0, 0, 2, 3, 3, 1],
        [55, 1, 0, 130, 262, 0, 1, 155, 1, 2, 2, 3, 3, 1],
```

```
        [59, 1, 0, 126, 218, 1, 1, 134, 0, 2, 2, 2, 7, 1],
        [60, 1, 0, 129, 196, 0, 1, 131, 1, 1, 2, 3, 3, 1],
        [54, 1, 0, 120, 188, 0, 1, 113, 0, 1, 2, 3, 3, 1],
        [57, 1, 0, 150, 276, 0, 0, 112, 1, 1, 2, 3, 7, 1],
        [61, 0, 2, 130, 330, 0, 1, 169, 0, 0, 2, 3, 3, 1],
    ]
    columns = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
               'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']
    df = pd.DataFrame(data, columns=columns)
    csv_path = os.path.join(OUTPUT_DIR, 'heart_disease.csv')
    df.to_csv(csv_path, index=False)
    print(f"  Heart disease dataset saved to: {csv_path}")
    print(f"  Total samples: {len(df)}")
    print(f"  Positive cases: {len(df[df['target'] == 1])}")
    print(f"  Negative cases: {len(df[df['target'] == 0])}")
    return df
def train_malaria_model():
    print("\n" + "="*50)
    print("TRAINING MALARIA MODEL")
    print("="*50)
    X, y = load_malaria_images()
    if len(X) == 0:
        print("No malaria data found!")
        return None
    model = MalariaModel()
    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(
```

```
        X, y, test_size=0.2, random_state=42, stratify=y
    )
     from sklearn.utils.class_weight import compute_class_weight
    class_weights = compute_class_weight('balanced', classes=np.unique(y_train),
y=y_train)
    class_weight_dict = dict(zip(np.unique(y_train), class_weights))
    print(f"\nClass weights: {class_weight_dict}")
    print(f"\nTraining on {len(X_train)} samples...")
    model.classifier.fit(
        X_train, y_train,
        epochs=10,
        batch_size=32,
        validation_split=0.2,
        class_weight=class_weight_dict,
        verbose=1
    )
    test_loss, test_accuracy = model.classifier.evaluate(X_test, y_test, verbose=0)
    print(f"\nMalaria Model Results:")
    print(f"  Test Accuracy: {test_accuracy * 100:.2f}%")
    print(f"  Test Loss: {test_loss:.4f}")
    model_path = os.path.join(BASE_DIR, 'malaria_model.keras')
    model.save_model(model_path)
    print(f"  Model saved to: {model_path}")
    model.trained = True
    return model
def train_pneumonia_model():
    print("\n" + "="*50)
```

```
    print("TRAINING PNEUMONIA MODEL")
    print("="*50)
    X, y = load_pneumonia_images()
    if len(X) == 0:
        print("No pneumonia data found!")
        return None
    model = PneumoniaModel()
    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
    from sklearn.utils.class_weight import compute_class_weight
    class_weights = compute_class_weight('balanced', classes=np.unique(y_train),
y=y_train)
    class_weight_dict = dict(zip(np.unique(y_train), class_weights))
    print(f"\nClass weights: {class_weight_dict}")
    print(f"\nTraining on {len(X_train)} samples...")
    model.classifier.fit(
        X_train, y_train,
        epochs=10,
        batch_size=32,
        validation_split=0.2,
        class_weight=class_weight_dict,
        verbose=1
    )
    test_loss, test_accuracy = model.classifier.evaluate(X_test, y_test, verbose=0)
    print(f"\nPneumonia Model Results:")
```

```
    print(f"  Test Accuracy: {test_accuracy * 100:.2f}%")
    print(f"  Test Loss: {test_loss:.4f}")
    model_path = os.path.join(BASE_DIR, 'pneumonia_model.keras')
    model.save_model(model_path)
    print(f"  Model saved to: {model_path}")
    model.trained = True
    return model
def train_heart_disease_model():
     print("\n" + "="*50)
    print("TRAINING HEART DISEASE MODEL")
    print("="*50)
    csv_path = os.path.join(OUTPUT_DIR, 'heart_disease.csv')
    if not os.path.exists(csv_path):
        print("Heart disease CSV not found!")
        return None
    df = pd.read_csv(csv_path)
    X = df.drop('target', axis=1).values
    y = df['target'].values
    print(f"Loaded {len(X)} samples from heart_disease.csv")
    model = HeartDiseaseModel()
    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
    print(f"\nTraining on {len(X_train)} samples...")
    model.train(X_train, y_train)
    from sklearn.metrics import accuracy_score, classification_report
```

```
    y_pred = model.model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    print(f"\nHeart Disease Model Results:")

    print(f"  Test Accuracy: {accuracy * 100:.2f}%")

    print("\nClassification Report:")

    print(classification_report(y_test, y_pred, target_names=['No Disease', 'Heart
Disease']))

    model_path = os.path.join(BASE_DIR, 'heart_disease_model.pkl')

    model.save_model(model_path)

    print(f"  Model saved to: {model_path}")

    model.trained = True

    return model

def train_diabetes_model():

     print("\n" + "="*50)

    print("TRAINING DIABETES MODEL")

    print("="*50)

    csv_path = os.path.join(OUTPUT_DIR, 'diabetes.csv')

    if not os.path.exists(csv_path):

        print("Diabetes CSV not found!")

        return None

    df = pd.read_csv(csv_path)

    X = df.drop('target', axis=1).values

    y = df['target'].values

    print(f"Loaded {len(X)} samples from diabetes.csv")

    print(f"  Positive cases: {np.sum(y == 1)}")

    print(f"  Negative cases: {np.sum(y == 0)}")

    model = DiabetesModel()
```

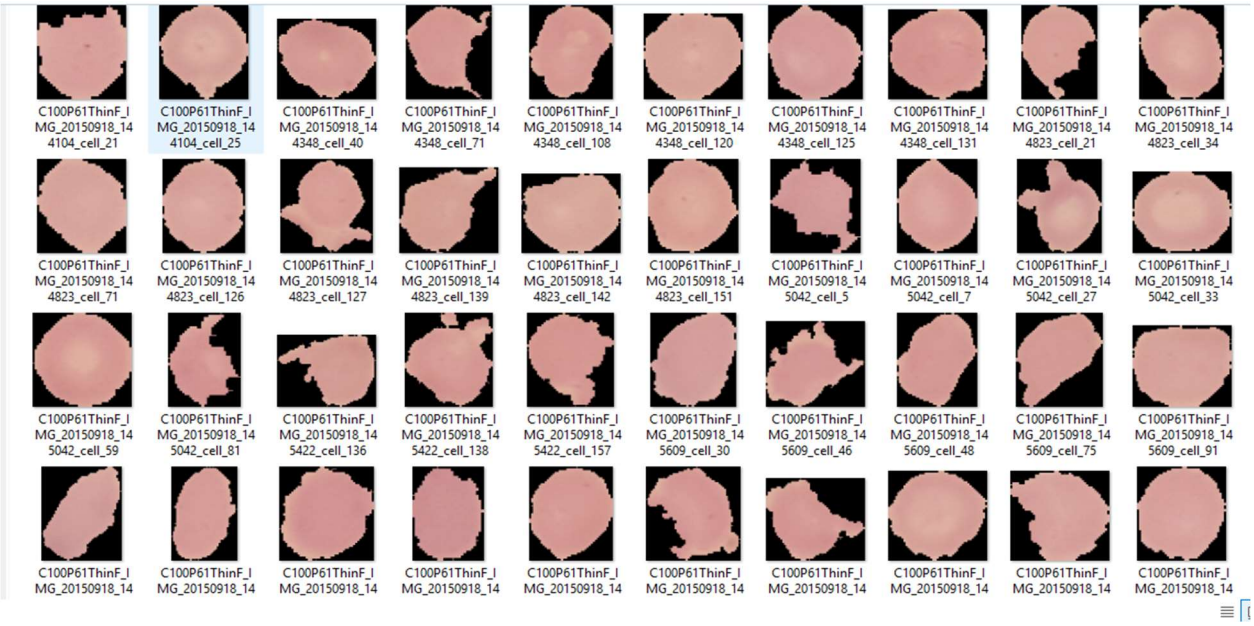
```
    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y )
    print(f"\nTraining on {len(X_train)} samples...")
    model.train(X_train, y_train)
    from sklearn.metrics import accuracy_score, classification_report
    y_pred = model.model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print(f"\nDiabetes Model Results:")
    print(f"  Test Accuracy: {accuracy * 100:.2f}%")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, target_names=['No Diabetes',
'Diabetes']))
    model_path = os.path.join(BASE_DIR, 'diabetes_model.pkl')
    model.save_model(model_path)
    print(f"  Model saved to: {model_path}")
    model.trained = True
    return model
if __name__ == "__main__":
    print("="*60)
    print("MULTI-DISEASE PREDICTION SYSTEM - MODEL TRAINING")
    print("="*60)
     create_heart_disease_csv()
     try:
        malaria_model = train_malaria_model()
    except Exception as e:
        print(f"Error training malaria model: {e}")
```

```
try:
    pneumonia_model = train_pneumonia_model()
except Exception as e:
    print(f"Error training pneumonia model: {e}")
try:
    heart_disease_model = train_heart_disease_model()
except Exception as e:
    print(f"Error training heart disease model: {e}")
try:
    diabetes_model = train_diabetes_model()
except Exception as e:
    print(f"Error training diabetes model: {e}")
print("\n" + "="*60)
print("TRAINING COMPLETE!")
print("="*60)
```

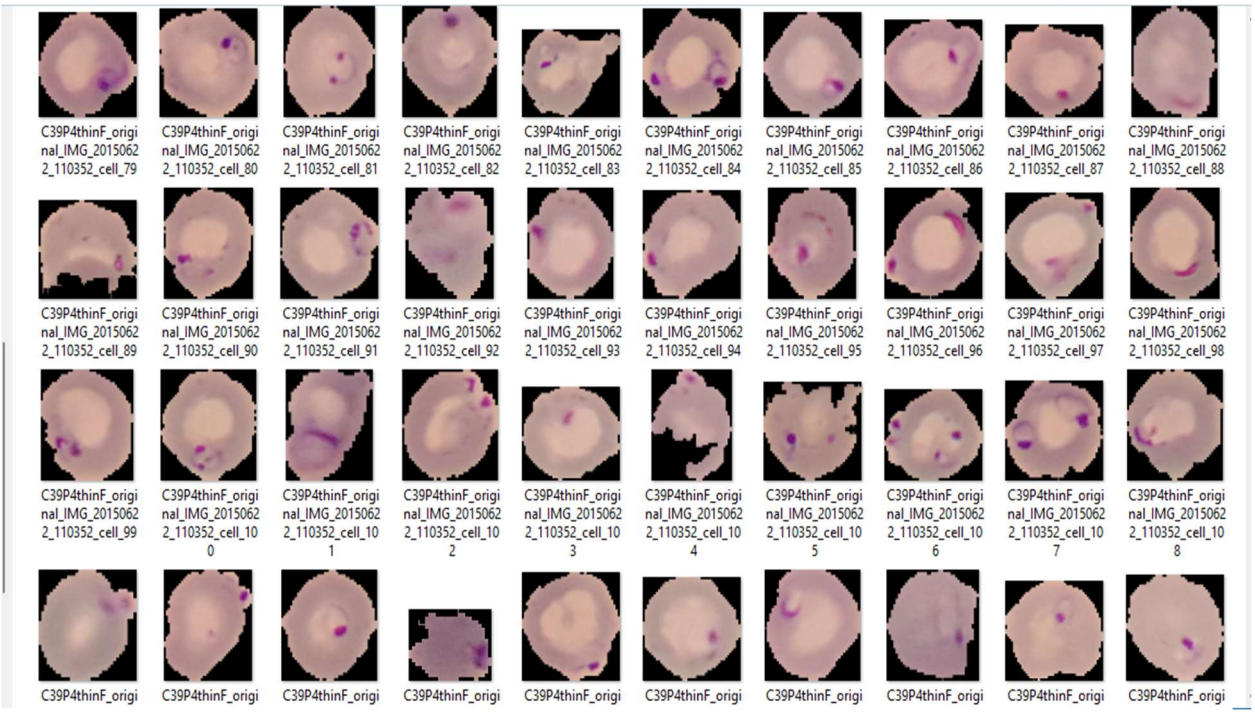
DATASET

Train:

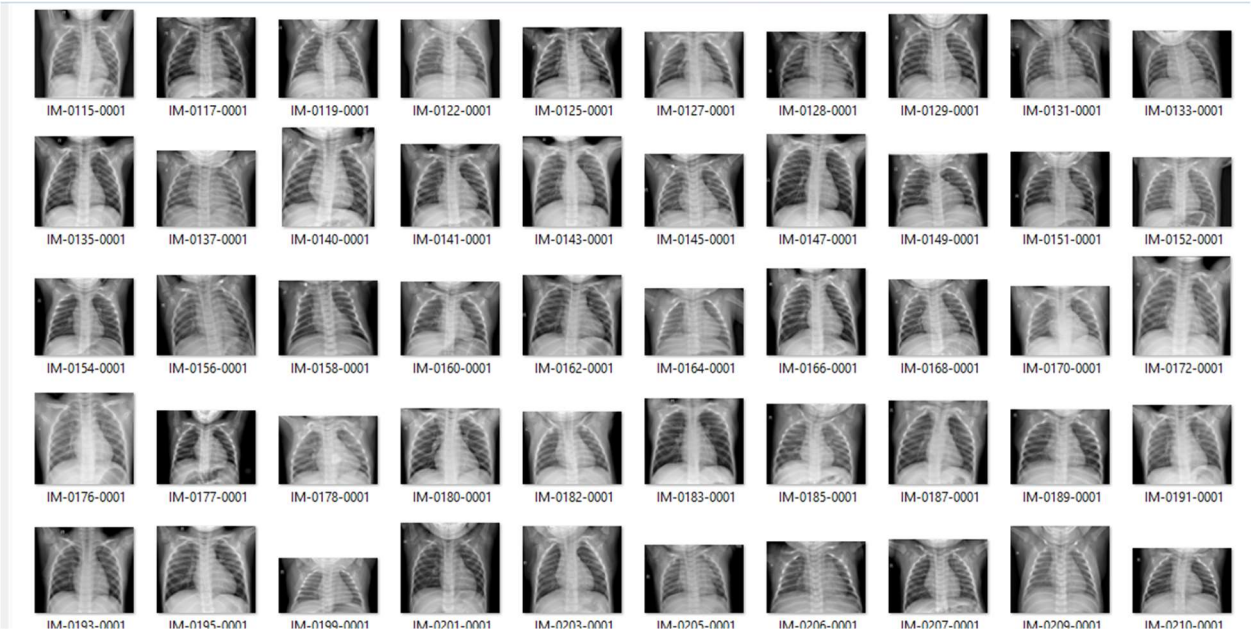
Malaria Detection Uninfected



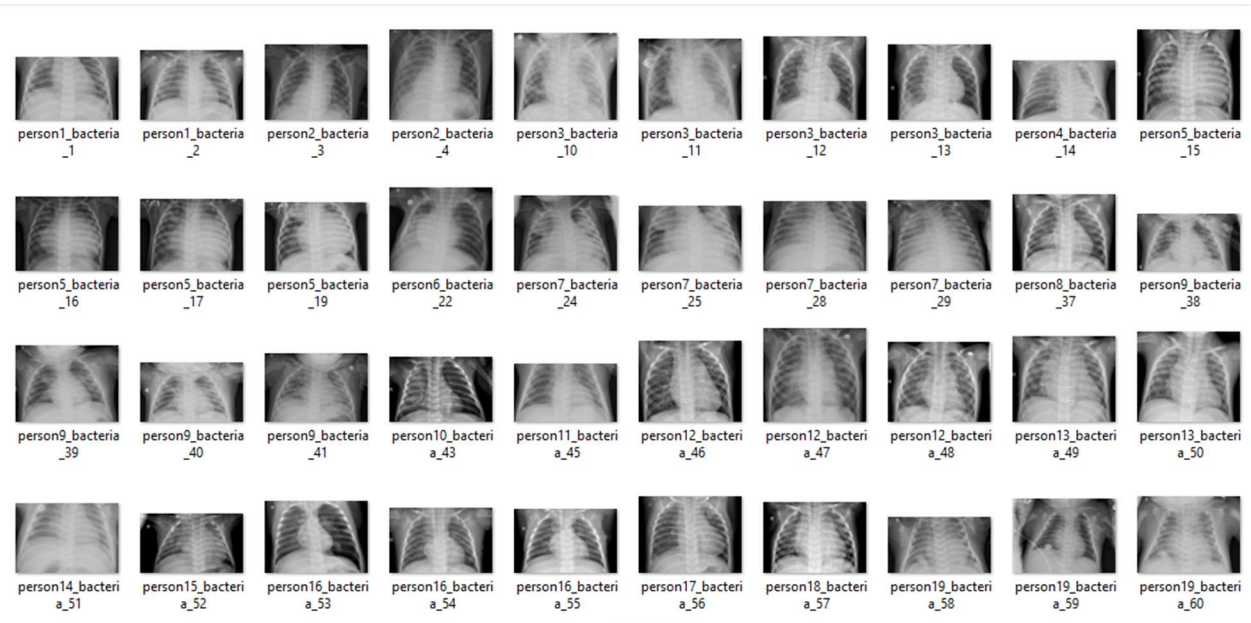
Malaria Detection Infected



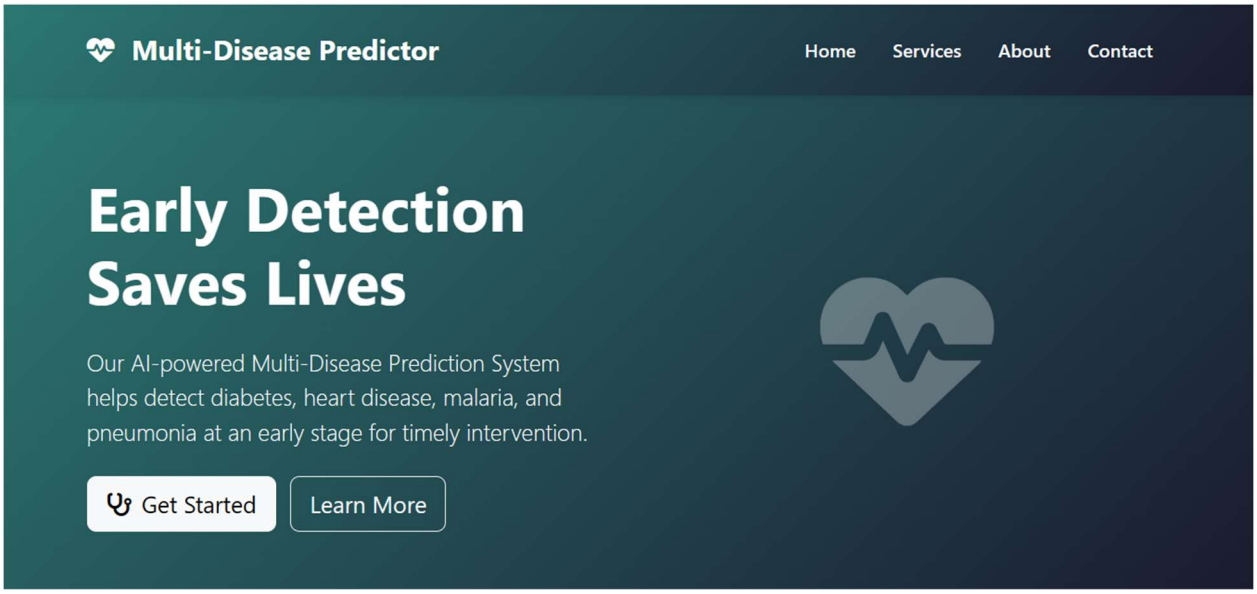
Pneumonia Detection Normal

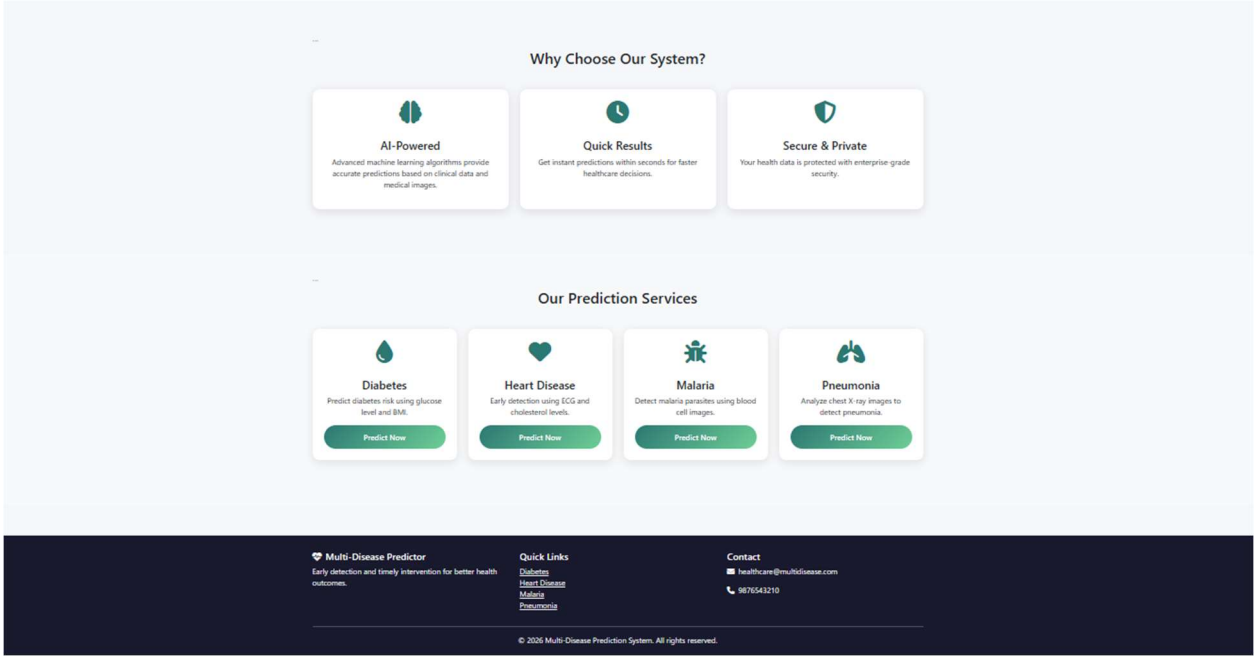


Pneumonia Detection Detected

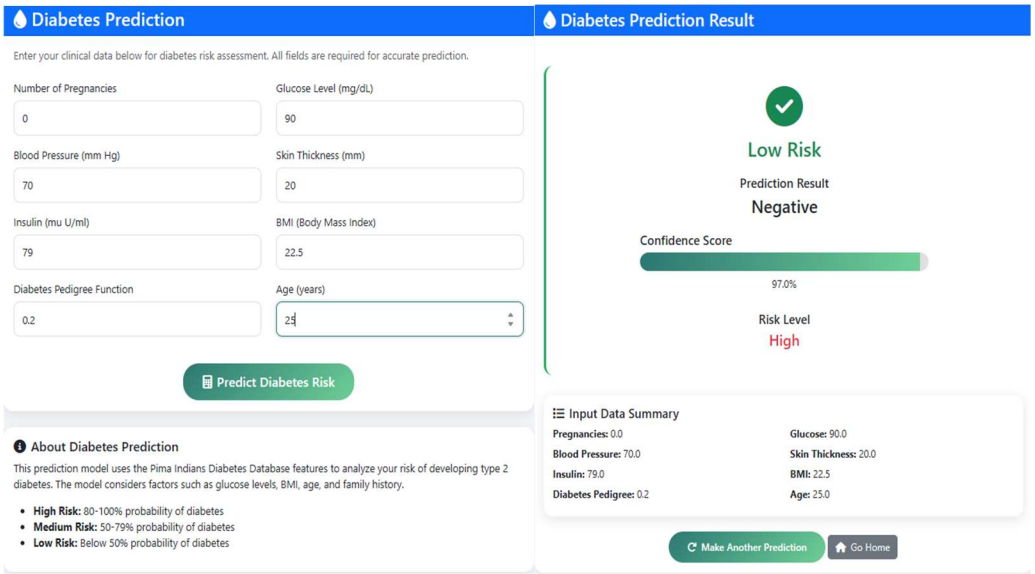


C. SAMPLE INPUT AND OUTPUT

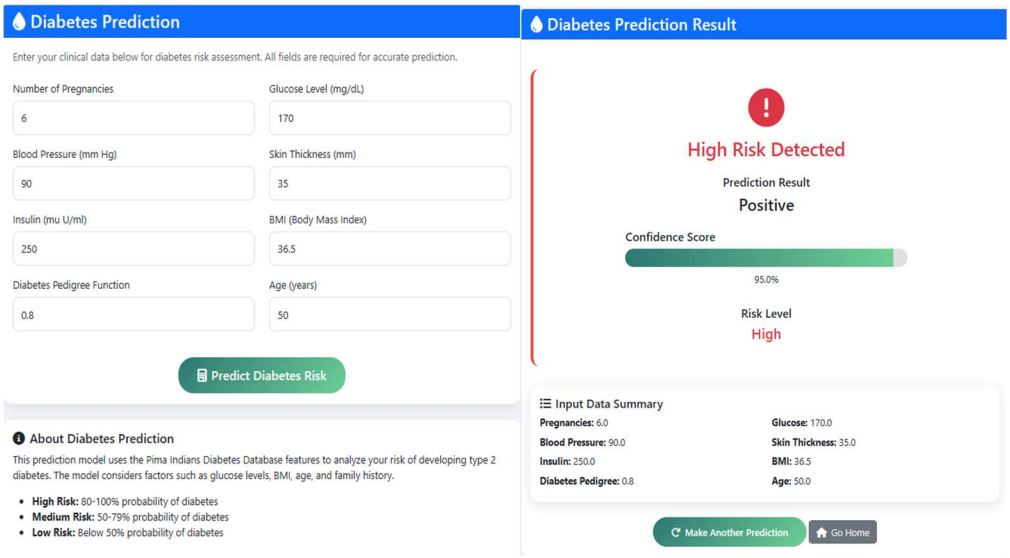




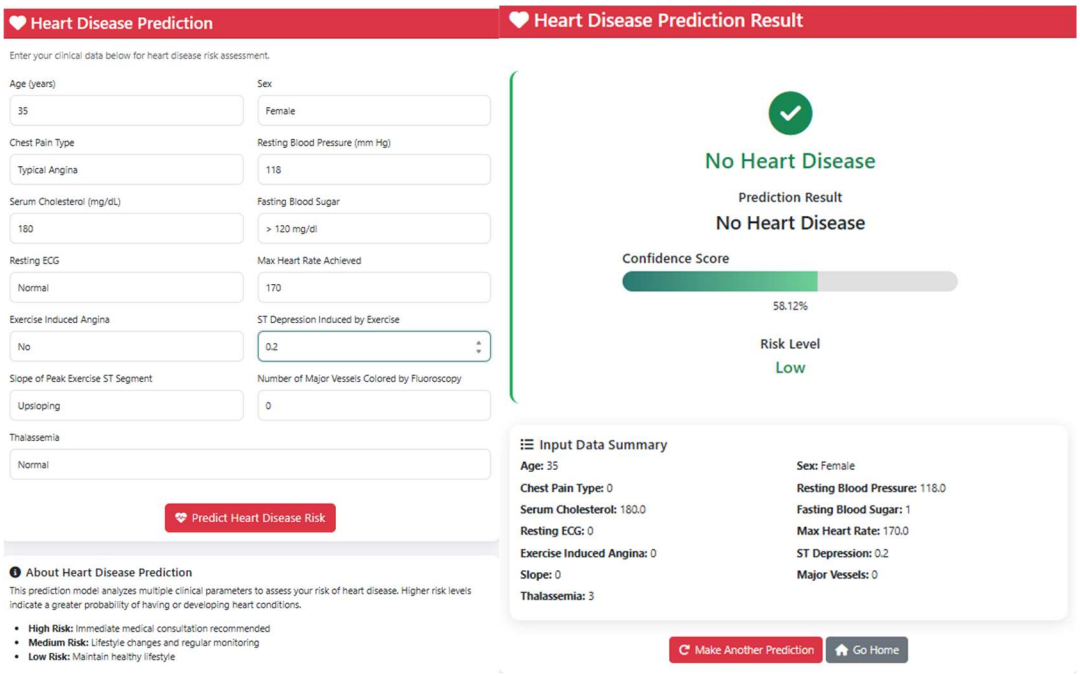
Diabetes Prediction Negative



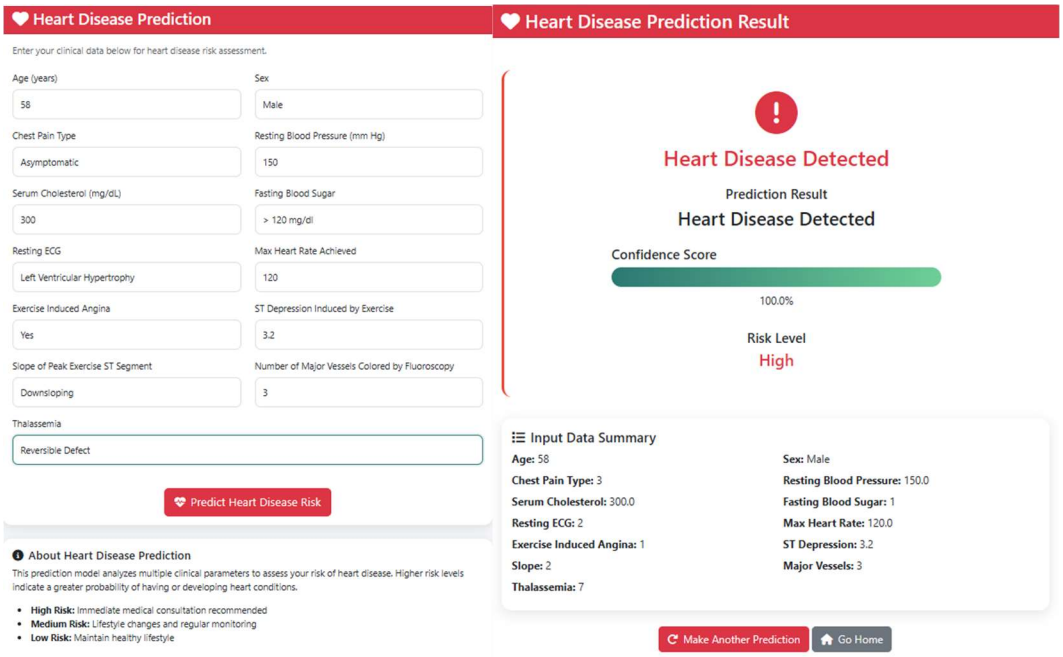
Diabetes Prediction Positive



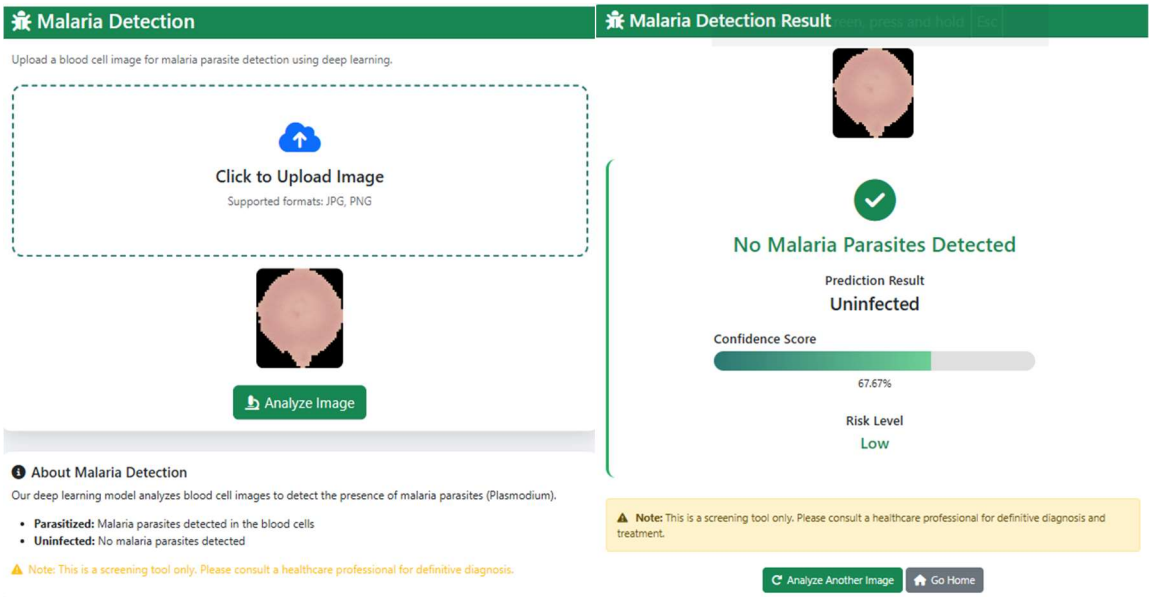
Heart Disease Prediction Negative



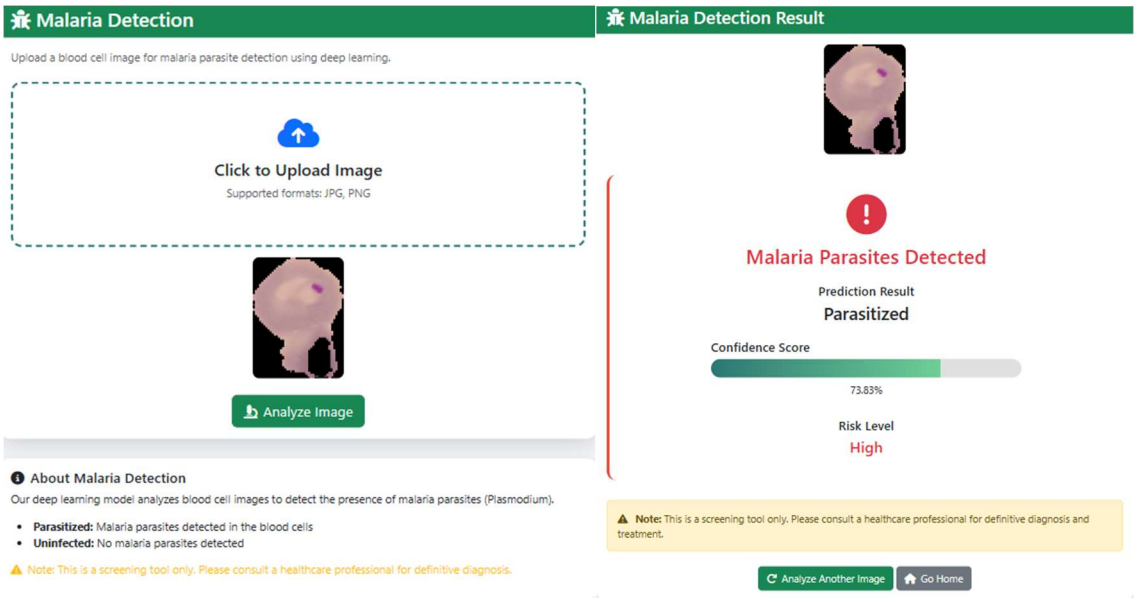
Heart Disease Prediction Positive



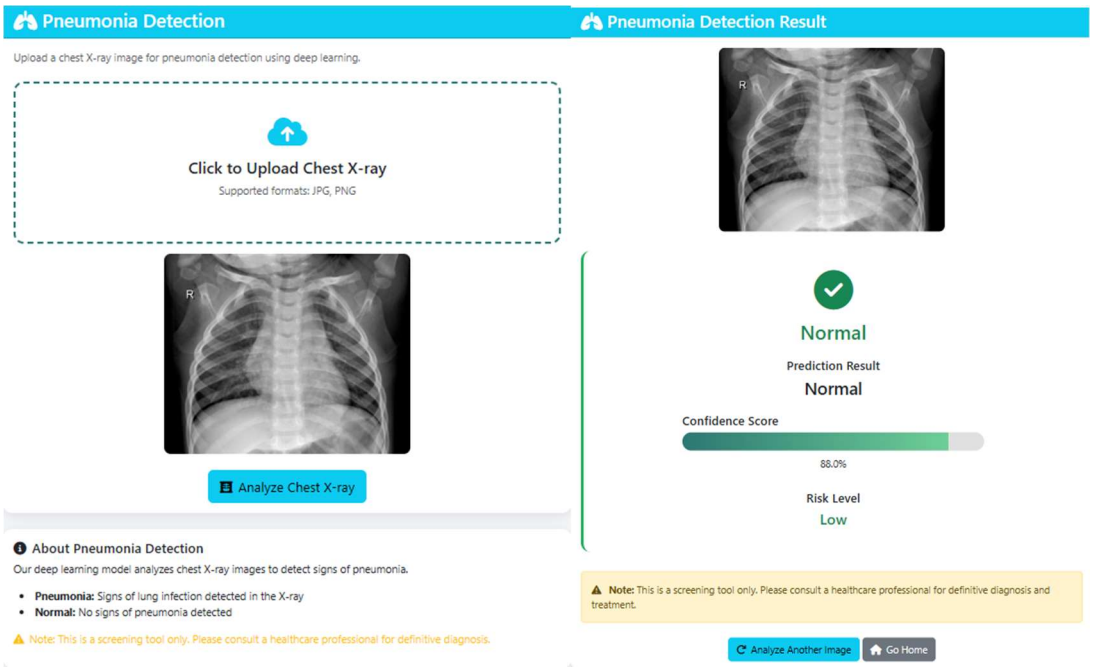
Malaria Detection Uninfected



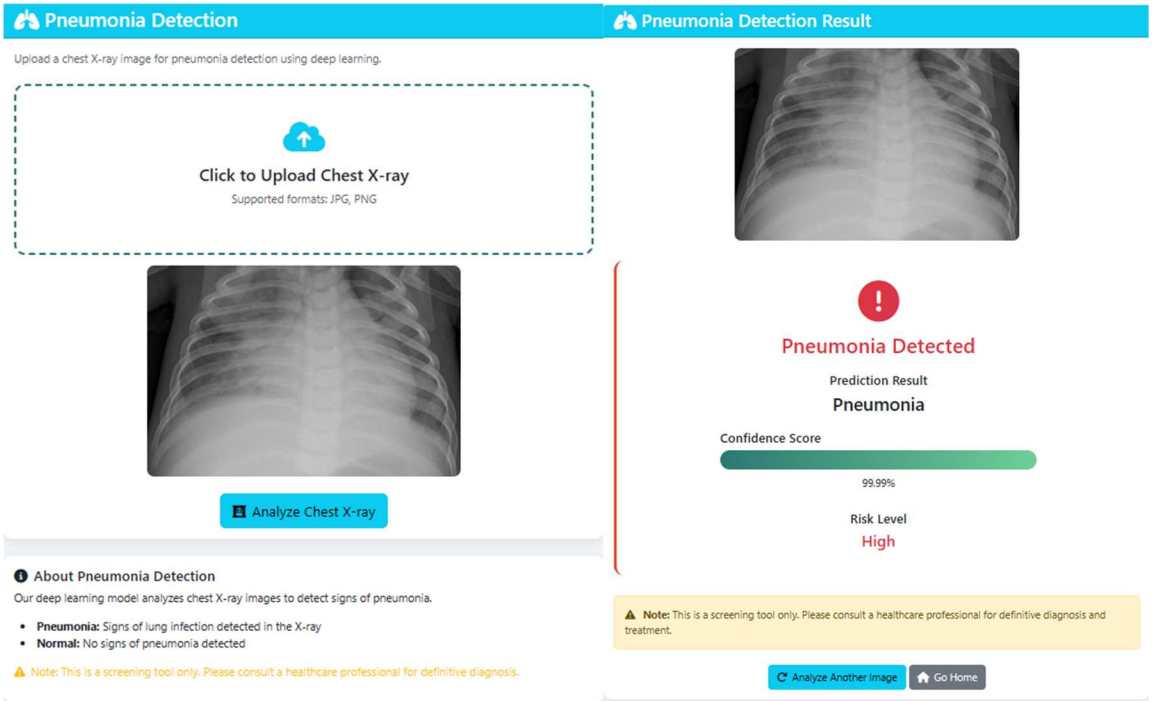
Malaria Detection Infected

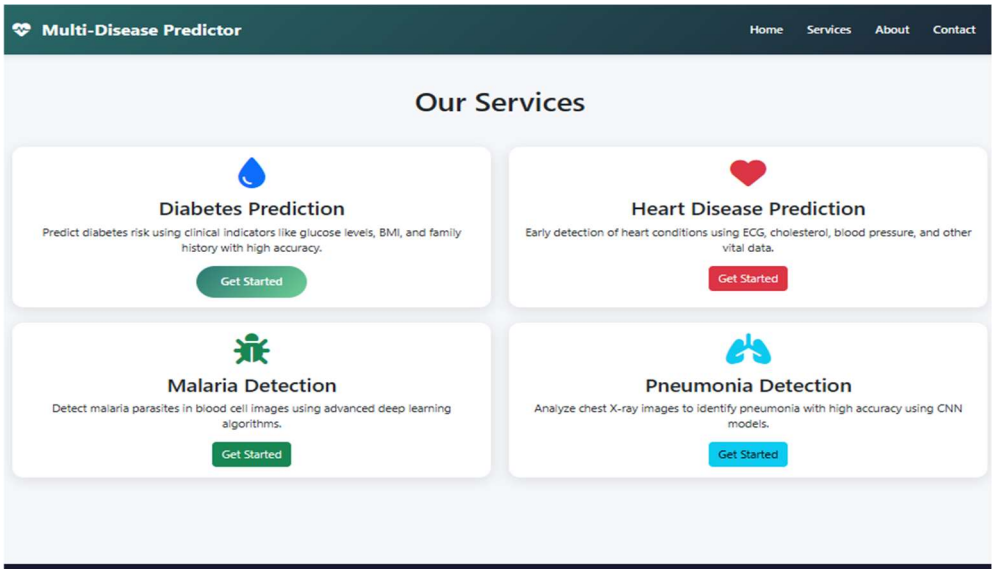


Pneumonia Detection Normal

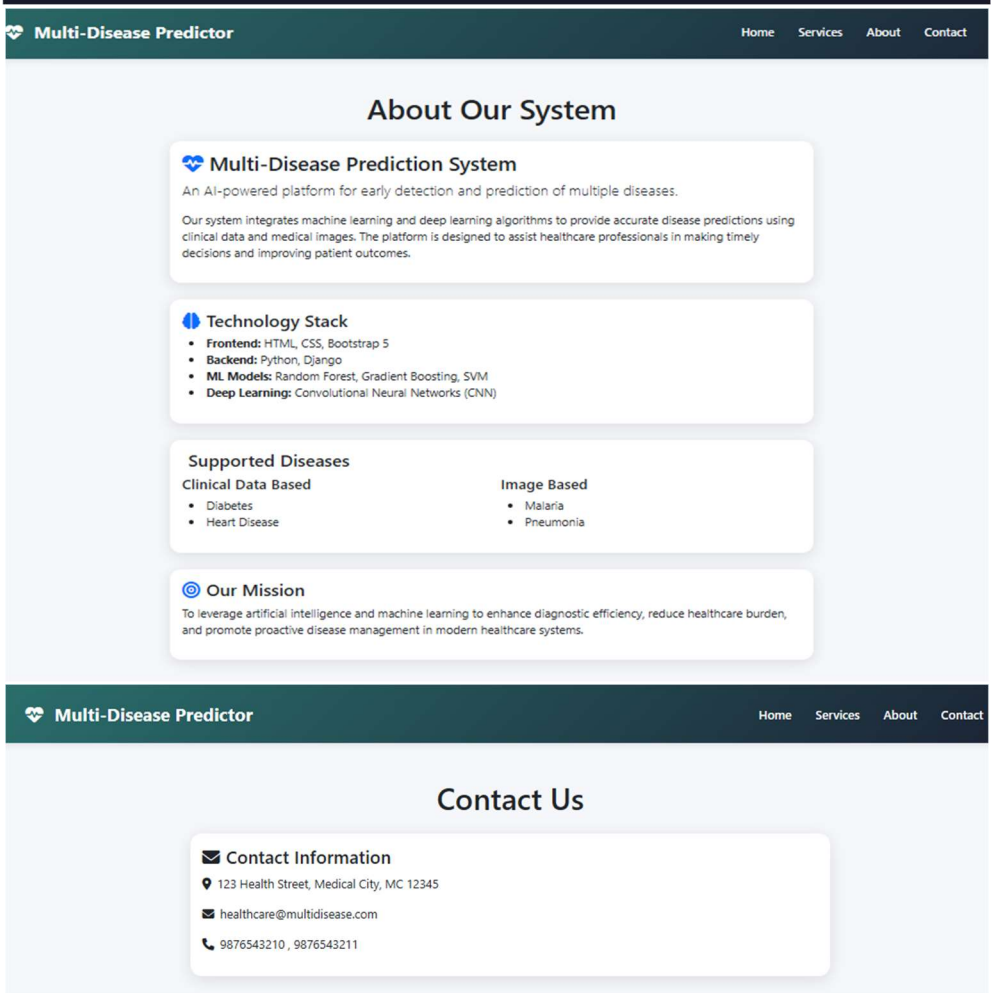


Pneumonia Detection Detected




Multi-Disease Predictor
Home
Services
About
Contact
Our Services
Diabetes Prediction
Predict diabetes risk using clinical indicators like glucose levels, BMI, and family history with high accuracy.
Get Started
Heart Disease Prediction
Early detection of heart conditions using ECG, cholesterol, blood pressure, and other vital data.
Get Started
Malaria Detection
Detect malaria parasites in blood cell images using advanced deep learning algorithms.
Get Started
Pneumonia Detection
Analyze chest X-ray images to identify pneumonia with high accuracy using CNN models.
Get Started

Multi-Disease Predictor
Early detection and timely intervention for better health outcomes.
Quick Links
Diabetes
Heart Disease
Malaria
Pneumonia
Contact
healthcare@multidisease.com
9876543210


Multi-Disease Predictor
Home
Services
About
Contact
About Our System
Multi-Disease Prediction System
An AI-powered platform for early detection and prediction of multiple diseases.
Our system integrates machine learning and deep learning algorithms to provide accurate disease predictions using clinical data and medical images. The platform is designed to assist healthcare professionals in making timely decisions and improving patient outcomes.
Technology Stack
Frontend: HTML, CSS, Bootstrap 5
Backend: Python, Django
ML Models: Random Forest, Gradient Boosting, SVM
Deep Learning: Convolutional Neural Networks (CNN)
Supported Diseases
Clinical Data Based
Diabetes
Heart Disease
Image Based
Malaria
Pneumonia
Our Mission
To leverage artificial intelligence and machine learning to enhance diagnostic efficiency, reduce healthcare burden, and promote proactive disease management in modern healthcare systems.
Multi-Disease Predictor
Home
Services
About
Contact
Contact Us
Contact Information
123 Health Street, Medical City, MC 12345
healthcare@multidisease.com
9876543210 , 9876543211